

Portfolioübersicht — Referenzprojekte

Marcos Sousa · Python Backend & AI Application Engineer · Buttenwiesen, Bayern

Die folgenden Referenzprojekte sind eigenständig entwickelt und öffentlich einsehbar. Sie belegen unmittelbar das angebotene Leistungsspektrum – RAG-Systeme, agentische Workflows, LLM-gestützte Anwendungen sowie die sichere Anbindung von KI-Agenten an Unternehmenssysteme – einschließlich der methodischen Qualitätssicherung und der regulatorischen Anforderungen an KI-Systeme. Diese Projekte ruhen zugleich auf einem soliden Fundament der Softwareentwicklung: rund zweieinhalb Jahre als Softwareentwickler mit Tech-Lead-Verantwortung und als technischer Ansprechpartner für Enterprise-Kunden bei der asioso GmbH – belegt durch ein Arbeitszeugnis. Quellcode und, soweit zutreffend, eine live nutzbare Demo stehen zur unmittelbaren Überprüfung bereit.

Autorisierung für KI-Agenten — MCP-Server mit OAuth 2.0 und OpenID Connect

Code: github.com/marcosfsousa/mcp-erp · Vollständig auf dem Rechner des Lesers lauffähig (Docker Compose)

Was es zeigt: Wie ein KI-Agent an ein Unternehmenssystem angebunden wird, ohne dabei die Rechte des Nutzers zu überschreiten – ein Model-Context-Protocol-Server über einem Beschaffungs- und Rechnungsprozess, bei dem jede einzelne Anfrage autorisiert wird und nicht die Verbindung.

- Server der aktuellen Protokoll-Revision (2026-07-28, ohne Verbindungsinitialisierung) über Streamable HTTP, zustandslos betrieben als zwei Replikate hinter nginx – die Zustandslosigkeit ist dadurch überprüfbar statt behauptet. Nachgewiesen gegen drei unabhängige Clients, darunter Anthropic's eigenen ausgelieferten Client in einer aufgezeichneten Sitzung.
- Vollständiger Authorization-Code-Flow gegen Keycloak: Client-Identifikation über ein öffentlich gehostetes Metadaten-Dokument (Client Identity Metadata Document) statt dynamischer Registrierung, PKCE, kein Client-Secret im gesamten Projekt, Prüfung der Token-Zielgruppe am Resource Server sowie lokale Signaturprüfung gegen den veröffentlichten Schlüsselsatz.
- Autorisierungsentscheidung pro Anfrage aus der Schnittmenge von erteiltem Scope und Rollen, die der Server selbst auflöst: Das Token ist die Obergrenze dessen, was eine Anwendung im Namen einer Person tun darf, und niemals eine Aussage darüber, was diese Person darf. Dieselbe Anfrage an denselben Endpunkt liefert je nach Token unterschiedliche Antworten – ein Werkzeug, für das die Berechtigung fehlt, erscheint gar nicht erst in der Werkzeugliste.
- Entscheidungsmatrix als Datenquelle: 33 Zeilen (Prinzipal × Werkzeug × Datensatz → erwartetes Ergebnis), davon 15 erlaubt und 18 abgelehnt über sieben unterschiedliche Ablehnungsgründe. Aus derselben Quelle entstehen die Tests, die Seed-Daten und die Tabelle in der Dokumentation – eine Zeile ändern heißt, den Test mitzuändern.
- Angriffskatalog mit 34 benannten Szenarien, davon 19 unter Angabe der konkreten Spezifikationsklausel, die sie verteidigen; jede Zeile steht in nachgewiesener Eins-zu-eins-Beziehung zu dem Test, der sie widerlegen würde. Ergänzt um ein Normenregister, das jede bewusste Abweichung von einem MUST oder SHOULD samt Begründung offenlegt.
- 533 automatisierte Tests und sieben CI-Prüfungen, darunter ein vollständiger Authorization-Code-Flow gegen eine laufende Instanz bei jedem Pull Request. Fünfzehn Architekturentscheidungen sind mit den jeweils verworfenen Alternativen dokumentiert.

Anwendungsbezug: Genau dieses Muster wird gebraucht, sobald ein KI-Assistent auf ein internes System zugreifen soll – ERP, CRM, Ticketsystem, Dokumentenablage. Die zentrale Frage ist dabei nie, ob der Agent technisch zugreifen kann, sondern in wessen Namen und mit welcher Obergrenze. Die hier gezeigte Trennung – Einwilligung des Nutzers, Scope als Delegationsgrenze, Rollen serverseitig aufgelöst – ist unmittelbar auf produktive Integrationen übertragbar und schafft zugleich die Nachweisbarkeit, die im Umgang mit personenbezogenen Daten ohnehin verlangt wird.

Stack: Python · FastAPI · Model Context Protocol (Revision 2026-07-28) · OAuth 2.0 · OpenID Connect · PKCE · Keycloak · PostgreSQL · Docker Compose · nginx · pytest · GitHub Actions

ScienceQ — Mehrsprachiges RAG-System (Wissensassistent für Videoinhalte)

Demo: scienceq.app · Code: github.com/marcosfsousa/project-ironhack-scienceq

Was es zeigt: Ein ausgeliefertes, live nutzbares System, das vier Kernbereiche in einem Projekt vereint – ein durchsuchbares Wissenssystem (RAG), agentische Orchestrierung, die methodische Qualitätssicherung und die regulatorische Umsetzung.

- End-to-end-RAG-System über einen mehrsprachigen Korpus (EN/ES/DE/FR/PT) aus 50 Wissenschafts-Transkripten: Ingestion, sprachensible Aufbereitung, Chunking, Vektorindex (Pinecone) sowie ein produktionsreifes FastAPI-Backend mit React/TypeScript-Single-Page-Application, betrieben auf Google Cloud Run, mit anklickbaren, zeitgestempelten Quellenangaben.
- Mehrsprachige semantische Suche auf Basis von Cohere-Embeddings mit nachgelagerter Rerank-Stufe, die überabgerufene Kandidaten auf die relevantesten verdichtet.
- Agentische Orchestrierung mit LangGraph für mehrstufige Retrieval- und Generierungsabläufe, inklusive regelbasiertem Intent-Routing ohne zusätzliche Modellkosten und einem Konversationsgedächtnis.
- Systematische Qualitätssicherung: 46-Fälle-Evaluationsset (darunter faktische, mehrsprachige und mehrstufige Fälle sowie 10 gezielt adversariale Fälle, die manuell geprüft werden), bewertet per LLM-as-Judge (GPT-4.1) und nachverfolgt über LangSmith – bester Messpunkt: Mittelwert 4,67/5 über die 33 automatisiert bewerteten Fälle; containerisiert und auf Google Cloud Run bereitgestellt – konkreter Beleg für das Versprechen messbar funktionierender Anwendungen statt bloßer Prototypen.
- Regulatorische Umsetzung (EU-KI-Verordnung, Art. 50): Die Transparenzpflichten für KI-Systeme mit direkter Nutzerinteraktion sind vor dem Geltungsbeginn am 2. August 2026 umgesetzt – KI-Hinweis auf der Startseite und an jeder Antwort sowie ein eigener Identitäts-Intent im Agenten-Router, der ohne Modellaufruf und in der Sprache des Nutzers wahrheitsgemäß über die KI-Natur des Systems informiert. Abgesichert durch Unit-Tests und fünf gezielt adversariale Evaluationsfälle; ergänzt um eine maschinenlesbare Herkunftskennzeichnung als Grundlage für Art. 50(2) sowie einen dokumentierten Prüfkatalog, der vor der Auslieferung bestimmter Funktionen eine erneute Art.-50-Bewertung erzwingt.

Anwendungsbezug: Dasselbe Muster – ein durchsuchbares, quellengestütztes Wissenssystem über einen mehrsprachigen Dokumentenbestand – ist unmittelbar auf unternehmensinterne Wissens- und Dokumentensysteme übertragbar und lässt sich auch unter erhöhten Datenschutzanforderungen umsetzen. Die regulatorische Vorarbeit ist dabei ebenso übertragbar: dieselben Transparenzpflichten nach Art. 50 treffen jedes kundenseitige KI-System mit direkter Nutzerinteraktion.

Stack: Python · FastAPI · React · TypeScript · LangChain · LangGraph · Cohere Embeddings & Rerank · Pinecone · Groq API · Google Cloud Run · nginx · Docker · LangSmith

JobScout — Autonome agentische Pipeline zur Stellenanalyse

Code: github.com/marcosfsousa/agentik-job-hunter

Was es zeigt: Demonstriert den angebotenen Bereich der agentischen Workflows (automatisierte mehrstufige KI-Prozesse) sowie das Nutzenversprechen – Produktionsreife statt Prototypen und ingenieurmäßige Solidität.

- Autonome mehrstufige Pipeline (Ingestion → Deduplizierung → Filterung → Embedding und Ranking → LLM-Bewertung → Auslieferung), vollständig automatisiert und ohne manuelle Eingriffe lauffähig.
- Asynchrone Ingestionsschicht mit austauschbaren Quell-Adaptern und deterministischer Vorverarbeitung, sodass die LLM-Bewertungsstufe sauber strukturierte Eingaben erhält, statt Fehler vorgelagerter Stufen auffangen zu müssen.
- Strukturiertes LLM-Bewertungsframework mit kalibrierter Bewertungslogik; idempotente, SQLite-gestützte Pipeline mit Deduplizierung und rückkopplungsbasiertem Re-Ranking.
- Rund 300 automatisierte Tests (pytest) zur Sicherung der Systemzuverlässigkeit; täglicher automatisierter Lauf über GitHub Actions – ein konkreter Nachweis produktionsnaher Engineering-Standards, die reine KI-Anwender ohne Engineering-Hintergrund typischerweise nicht abdecken.

Anwendungsbezug: Das zugrunde liegende Muster – die automatisierte, mehrstufige Filterung und Bewertung großer Mengen unstrukturierter Eingaben – ist auf zahlreiche geschäftliche Aufgaben übertragbar, etwa die Vorsortierung von Dokumenten, Anfragen oder Bewerbungen.

Stack: Python · asyncio · httpx · Pydantic · SQLite · sentence-transformers · OpenAI API · pytest · GitHub Actions